Algorithm Analysis

Adopted from Clifford A. Shaffer. Data Structures and Algorithm Analysis. Chapter 3

Introduction

- Should I buy a new program from vendor X or vendor Y?
- Why a particular program is slow?
  - is it badly implemented?
  - is it solving a hard problem?
- Questions like these ask us to consider the difficulty of a problem, and the relative efficiency of the approaches used for solving a problem.

Algorithm Efficiency

- There are often many approaches (algorithms) to solve a problem.
- How do we choose between them?
- At the heart of computer program design are two (sometimes conflicting) goals.
  - To design an algorithm that is easy to understand, code, debug.
  - To design an algorithm that makes efficient use of the computer's resources.

How to Measure Efficiency?

1. Empirical comparison (run programs)

Limitations:

- Programming & Testing effort
- Better written (Biasing)
- Unfair test cases
- Both doesn’t meet budget

How to Measure Efficiency?

2. Asymptotic Analysis (estimate the resource consumption of an algorithm)

- Critical resources:
  - Running Time
  - Space (RAM, Disk)

- Factors affecting running time:
- Hardware
- Number of users & resources
- Programming Language
- Coding & Compiler

How to Measure Efficiency? Contd.

- For most algorithms, running time depends on basic operations & the “size” of the input.

- Key metric: The number of **basic operations** required by the algorithm to process an input of a certain **size.**

- Time of a basic operation does not depend on the particular values of its operands(add/compare)

- Running time is expressed as **T**(*n*) for some function **T** on input size *n*.

Examples of Growth Rate

- Example 1: largest value sequential search algorithm.

```
/** @return Position of largest value in "A" */
int largest(int A[], int n) {
  int currlarge = 0;  // Position of largest
  for (int i=1; i<n; i++)
    if (A[currlarge] < A[i])
       currlarge = i; // Remember pos
  return currlarge;   // Return largest pos
}
```

Running Time : T(n) = cn

n : #of comparisons

c : amount of time required to compare two integers

This indicates the growth rate for the running time of the largest value sequential search algorithm. (The cost of the algorithm as the input size increases)

Examples (cont)

- Example 2: Assignment statement.

  $T(n) = c_1$ constant running time

- Example 3:

```
sum = 0;
for (i=1; i<=n; i++)
   for (j=1; j<=n; j++)
      sum++;
}
```

$T(n) = c_2 n^2$

n^2 : # of increments operations; c_2 increment time

Growth Rate

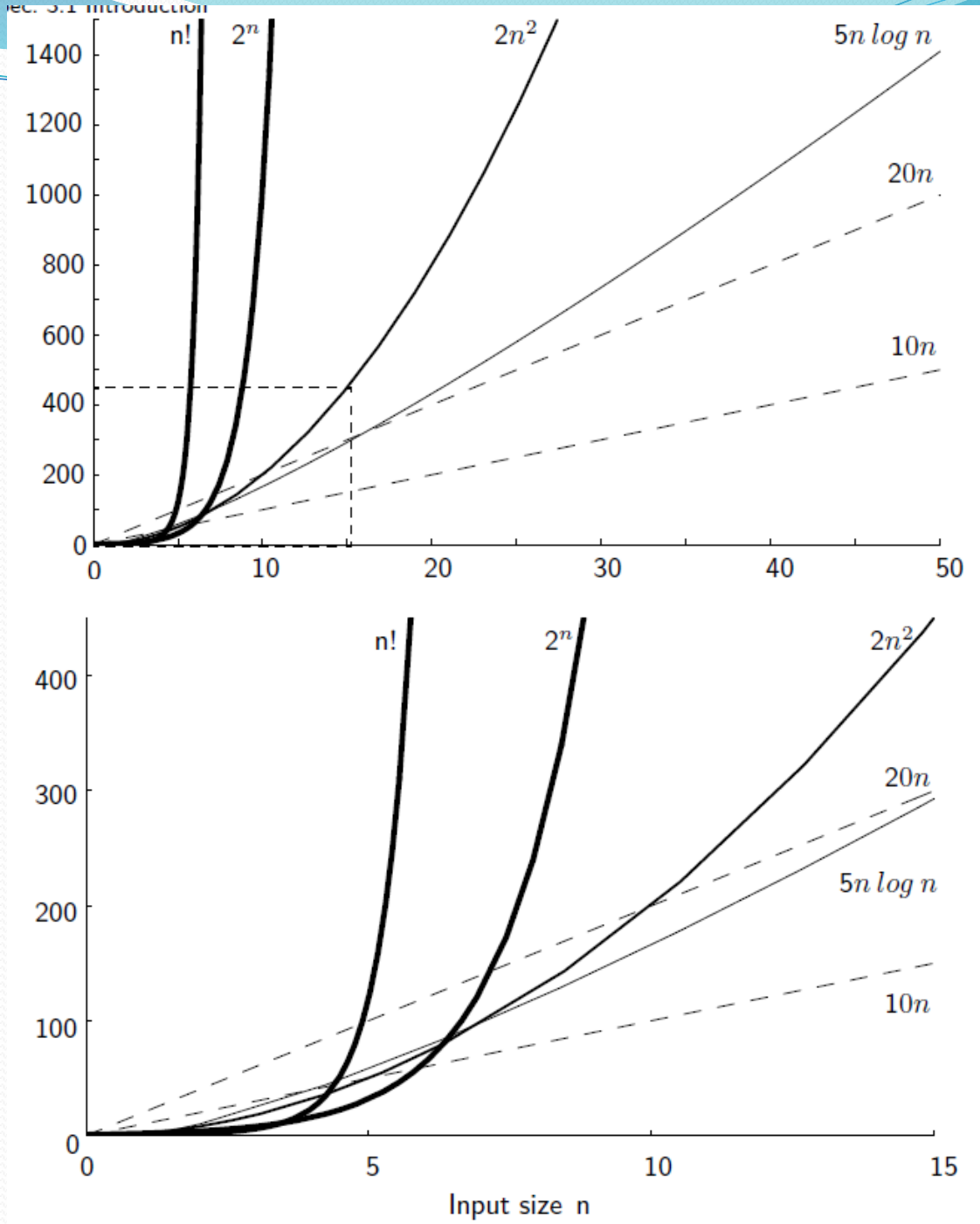
Algorithm growth rate: is the rate at which the cost of the algorithm grows as the size of its input grows.

cn: linear growth rate.

n^2 : quadratic

2^n : exponential

n! : exponential



Growth Rate

n	$\log \log n$	$\log n$	n	$n \log n$	n^2	n^3	2^n
16	2	4	2^4	$4 \cdot 2^4 = 2^6$	2^8	2^{12}	2^{16}
256	3	8	2^8	$8 \cdot 2^8 = 2^{11}$	2^{16}	2^{24}	2^{256}
1024	≈ 3.3	10	2^{10}	$10 \cdot 2^{10} \approx 2^{13}$	2^{20}	2^{30}	2^{1024}
64K	4	16	2^{16}	$16 \cdot 2^{16} = 2^{20}$	2^{32}	2^{48}	2^{64K}
1M	≈ 4.3	20	2^{20}	$20 \cdot 2^{20} \approx 2^{24}$	2^{40}	2^{60}	2^{1M}
1G	≈ 4.9	30	2^{30}	$30 \cdot 2^{30} \approx 2^{35}$	2^{60}	2^{90}	2^{1G}

Figure 3.2 Costs for growth rates representative of most computer algorithms.

Best, Worst, Average Cases

- In some problems, not all inputs of a given size take the same time to run.
- Sequential search for *K* in an array of *n* integers:
  - Begin at first element in array and look at each element in turn until *K* is found
- Best case: The first element (cost=1)
- Worst case: The last element (cost =n)
- Average case: n/2

Which Analysis to Use?

- Best case might happen only rarely.
- Average time: The typical behaviour of the algorithm on inputs of size n. It appears to be the fairest measure, but it may be difficult to determine (ex. Data Distribution is unknown)
- The worst case: for certain the algorithm must perform at least that well. important for real-time applications (Traffic control Systems)

Faster Computer or Algorithm?

Suppose we buy a computer 10 times faster.

n : size of input that can be processed in one second on old computer

n' : size of input that can be processed in one second on new computer

f(n)	n	n′	Change	n′/n
$10n$	1000	10,000	$n' = 10n$	10
$20n$	500	5000	$n' = 10n$	10
$5n \log n$	250	1842	$\sqrt{10}n < n' < 10n$	7.37
$2n^2$	70	223	$n' = \sqrt{10}n$	3.16
2^n	13	16	$n' = n + 3$	−−

Case Study

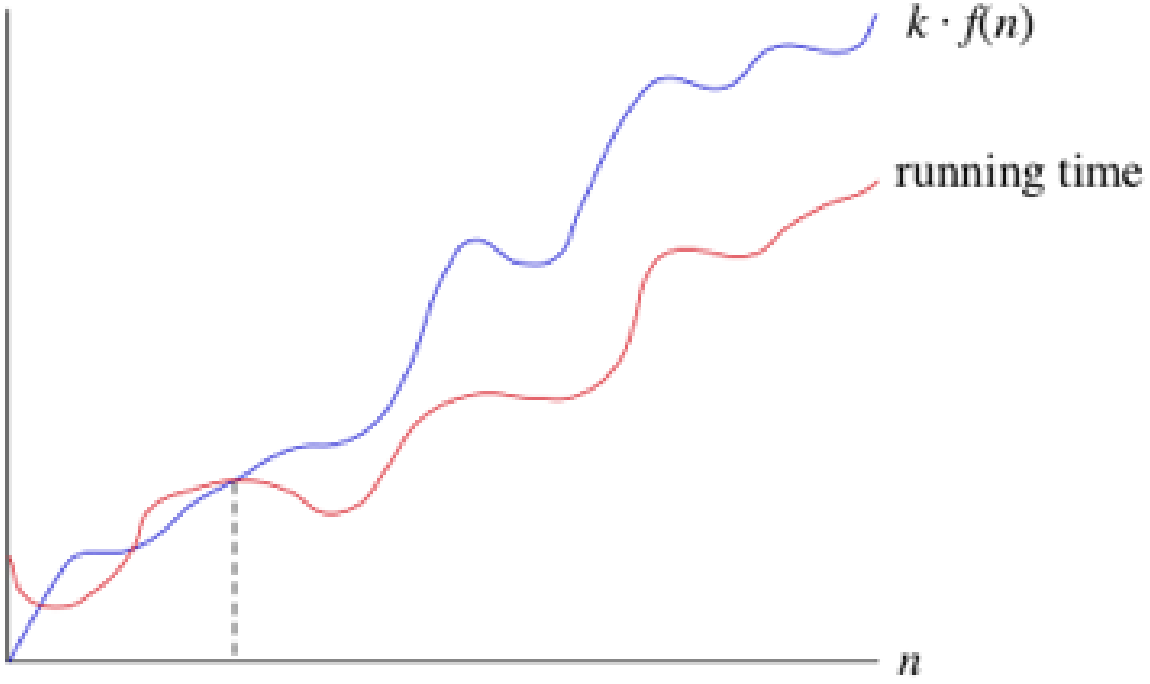
- An algorithm with running time T(n) = n2 requires 1024 * 1024 =1, 048, 576 time steps for an input of size n = 1024.
- An algorithm with running time T(n) = n log n requires 1024 * 10 = 10, 240 time steps for an input of size n = 1024.
- Enhancement: 1, 048, 576/ 10, 240=102.4

Asymptotic Analysis

- The study of an algorithm as the input size "gets big" or reaches a limit (in the calculus sense).
- Ignore all constant factors
- A form of "back of the envelope" estimation for algorithm resource consumption.
- It provides a simplified model of the running time or other resource needs of an algorithm. It helps you understand the behavior of your algorithms.

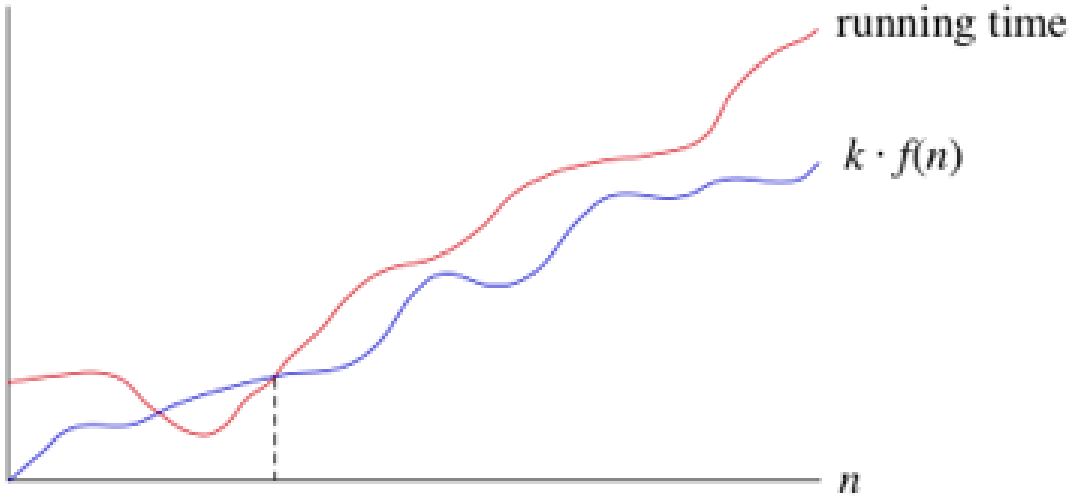
Big-O notation
Asymptotically Upper Bound

$O(f(n))$:



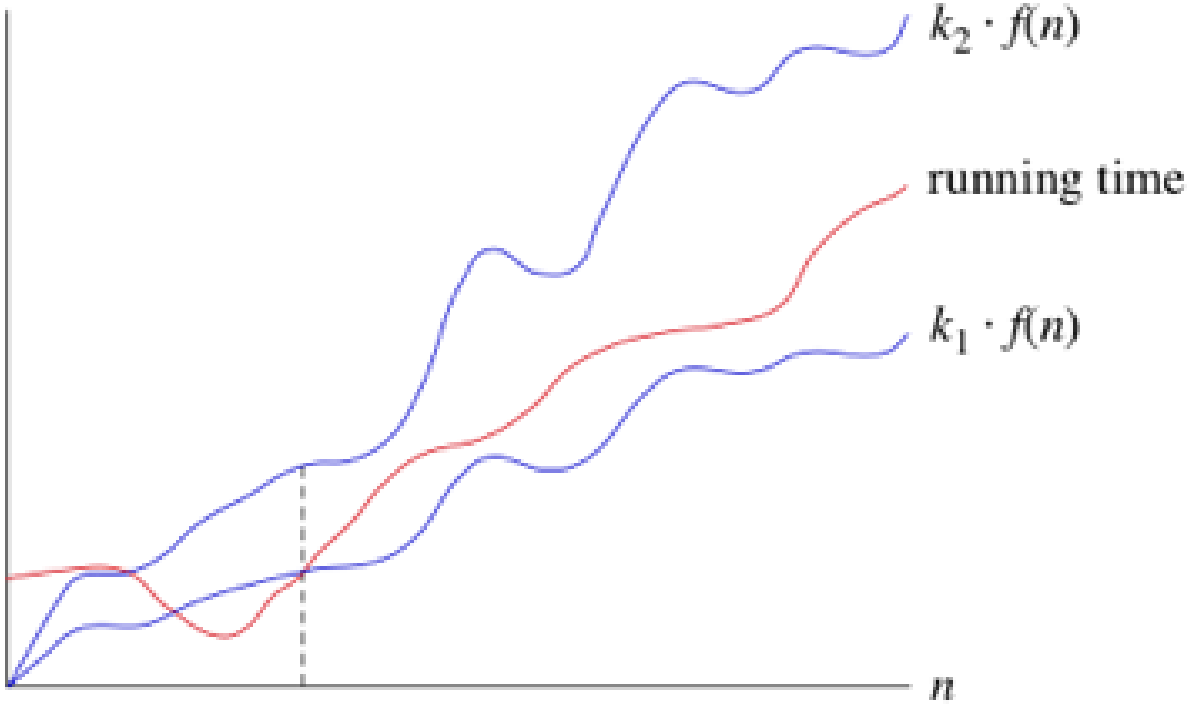
Big-Ω notation
Asymptotically Lower Bound

$\Omega(f(n))$:



Big-Θ notation
Asymptotically Tight Bound

$\Theta(f(n))$ for some function $f(n)$:



Asymptotic Analysis: Big-oh O

- Definition:

For **T**(*n*) a non-negatively valued function, **T**(*n*) is in the set O(*f*(*n*)) if there exist two positive constants *c* and n_0 such that **T**(*n*) <= *cf*(*n*) for all $n > n_0$.

- Meaning: For all data sets big enough (i.e., $n>n_0$), the algorithm always executes in less than *cf*(*n*) steps in the worst case.

Big-oh Notation (cont)

- Big-oh notation indicates an upper bound.
- Example: If $\mathbf{T}(n) = 3n^2$ then $\mathbf{T}(n)$ is in $O(n^2)$.
- Look for the tightest upper bound:
  - While $\mathbf{T}(n) = 3n^2$ is in $O(n^3)$, we prefer $O(n^2)$.

Big-Oh Examples

- Example 1: Finding value *X* in an array (average cost).

  Then **T**(*n*) = $c_s n/2$.

  For all values of $n > 1$, $c_s n/2 <= c_s n$.

  Therefore, the definition is satisfied for *f(n)*=*n*, $n_0 = 1$, and $c = c_s$.

  Hence, **T**(*n*) is in O(*n*).

Big-Oh Examples (2)

- Example 2: Suppose **T**(n) = $c_1n^2 + c_2n$, *where* c_1 and c_2 are positive.

  $c_1n^2 + c_2n <= c_1n^2 + c_2n^2 <= (c_1 + c_2)n^2$ for all $n > 1$.

  Then T(n) <= cn^2 whenever $n > n_0$, for $c = c_1 + c_2$ and $n_0 = 1$.

  Therefore, **T**(n) is in O(n^2) by definition.

- Example 3: **T**(n) = c. Then **T**(n) is in O(1).

A Common Misunderstanding

- "The best case for my algorithm is n=1 because that is the fastest." WRONG!
- Big-oh refers to a growth rate as n grows to ∞.
- Best case is defined is the cheapest.

Big-Omega Ω

- Definition:

For **T**(*n*) a non-negatively valued function, **T**(*n*) is in the set Ω(*g*(*n*)) if there exist two positive constants *c* and n_0 such that **T**(*n*) >= *cg*(*n*) for all $n > n_0$.

- Meaning: For all data sets big enough (i.e., $n > n_0$), the algorithm always requires more than *cg*(*n*) steps.

- <u>Lower bound.</u>

Big-Omega Example

- $\mathbf{T}(n) = c_1 n^2 + c_2 n$.

  $c_1 n^2 + c_2 n >= c_1 n^2$ for all $n > 1$.

  $\mathbf{T}(n) >= cn^2$ for $c = c_1$ and $n_0 = 1$.

  Therefore, $\mathbf{T}(n)$ is in $\Omega(n^2)$ by the definition.

- We want the greatest lower bound.

- For sequential search;

  $\mathbf{T}(n) = cn$ $\Omega(n)$

Theta Notation

- When big-Oh and Ω coincide, we indicate this by using Θ (big-Theta) notation.
- Definition: An algorithm is said to be in $\Theta(h(n))$ if it is in $O(h(n))$ and it is in $\Omega(h(n))$.
- sequential search algorithm is both
  - O(n) and Ω(n) in the average case,
  - Θ(n) in the average case.

A Common Misunderstanding

- Confusing worst case with upper bound.
- Upper bound refers to a growth rate.
- Worst case refers to the worst input from among the choices for possible inputs of a given size (Consumption of resources is the highest)

Simplifying Rules

1. If f(n) is in O(g(n)) and g(n) is in O(h(n)), then f(n) is in O(h(n)).
2. If f(n) is in O(kg(n)) for some constant k > 0, then f(n) is in O(g(n)).
3. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $(f_1 + f_2)(n)$ is in $O(\max(g_1(n), g_2(n)))$.
4. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $f_1(n)f_2(n)$ is in $O(g_1(n)g_2(n))$.

Classifying Functions

Given *f(n) & g(n)*. Which grows faster?

$$\lim_{n \to \infty} \frac{f(n)}{g(n)}.$$

If the limit goes to ∞, then $f(n)$ is in $\Omega(g(n))$ because $f(n)$ grows faster. If the limit goes to zero, then $f(n)$ is in $\mathrm{O}(g(n))$ because $g(n)$ grows faster. If the limit goes to some constant other than zero, then $f(n) = \Theta(g(n))$ because both grow at the same rate.

Classifying Functions-Example

Example 3.8 If $f(n) = 2n \log n$ and $g(n) = n^2$, is $f(n)$ in $O(g(n))$, $\Omega(g(n))$, or $\Theta(g(n))$? Because

$$\frac{n^2}{2n \log n} = \frac{n}{2 \log n},$$

$$\lim_{n \to \infty} \frac{n^2}{2n \log n} = \infty$$

because n grows faster than $2 \log n$. Thus, n^2 is in $\Omega(2n \log n)$.

Time Complexity Examples (1)

- Example 3.9: **`a = b;`**

  This assignment takes constant time, so it is **Θ(1)**.

- Example 3.10:

```
sum = 0;
for (i=1; i<=n; i++)
   sum += n;
```

Θ(n)

Time Complexity Examples (2)

- Example 3.11:

```
sum = 0;
for (j=1; j<=n; j++)
   for (i=1; i<=j; i++)
      sum++;
for (k=0; k<n; k++)
   A[k] = k;
```

$\Theta(n^2)$

Time Complexity Examples (3)

- Example 3.12:

```
sum1 = 0;
for (i=1; i<=n; i++)
   for (j=1; j<=n; j++)
      sum1++;

sum2 = 0;
for (i=1; i<=n; i++)
   for (j=1; j<=i; j++)
      sum2++;
```

$\Theta(n^2)$

Time Complexity Examples (4)

- Example 3.13:

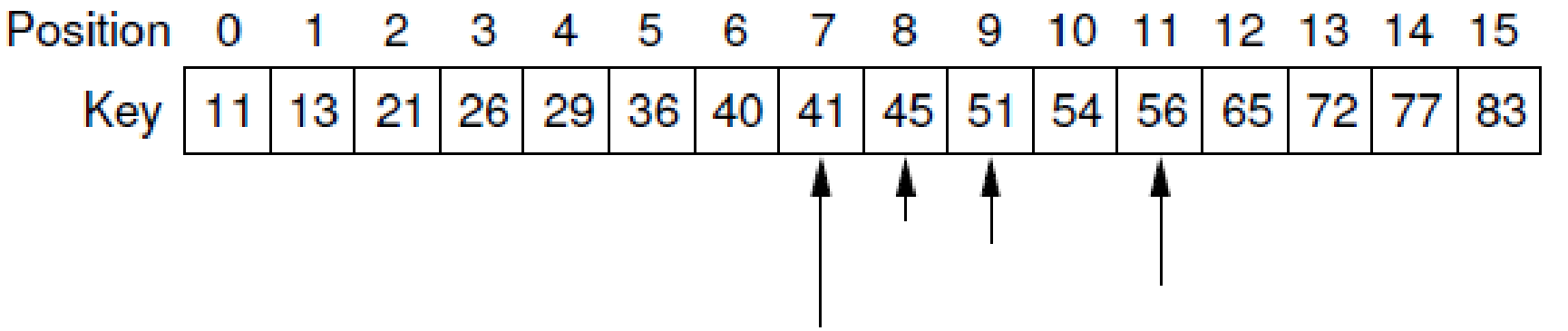
```
sum1 = 0;
for (k=1; k<=n; k*=2)
   for (j=1; j<=n; j++)
      sum1++;

sum2 = 0;
for (k=1; k<=n; k*=2)
   for (j=1; j<=k; j++)
      sum2++;
```

Other Control Statements

- **`while`** loop: Analyze like a **`for`** loop.
- **`if`** statement: Take greater complexity of **`then`**/**`else`** clauses.
- **`switch`** statement: Take complexity of most expensive case.
- Subroutine call: Complexity of the subroutine.

Binary Search



Binary Search(Do your own)

```
/* return The position of an element in sorted array A with
     value k.  If k is not in A, return -1. */
int binary(int[] A, int k) {
  int lower = 0, upper = A.length -1;
     // Set lower and upper limits of array
 while (lower<=upper) {
     int mid = (lower + upper)/2; // middle
     if (k < A[mid]) upper = mid-1;
                              //In left half
     else if (k > A[mid]) lower = mid+1;
                              //In right half
     else  return mid; //Found
  }
  return -1; // Not found
}
```

Analyzing Problems

- Upper bound: Upper bound of best known algorithm.
- Lower bound: Lower bound for every possible algorithm.

Analyzing Problems: Example

- May or may not be able to obtain matching upper and lower bounds.
- Example of imperfect knowledge: Sorting

  1. Cost of I/O: $\Omega(n)$.
  2. Bubble or insertion sort: $O(n^2)$ in worst case .
  3. A better sort (Quicksort, Mergesort, Heapsort, etc.): $O(n \log n)$.
  4. We prove later that sorting is in $\Omega(n \log n)$.

Space/Time Tradeoff Principle

- **One can often reduce time if one is willing to sacrifice space, or vice versa.**
  - Encoding or packing information
    - Boolean flags
  - Table lookup
    - Factorials
    - sine
- Disk-based Space/Time Tradeoff Principle:

The smaller you make the disk storage requirements, the faster your program will run.

Space Complexity

- Space complexity can also be analyzed with asymptotic complexity analysis.

  Time: Algorithm

  Space: Data Structure

- Example : What are the space requirements for an array of ***n*** integers?

  If each integer requires ***c*** bytes, then the array requires ***cn*** bytes, which is **Θ (n).**